

I'm implementing the CSEC 201 RFMP sockets project. My teammate already wrote the unsecured client/server (nima_client.py and nima_server.py). My job is to add the encrypted communication path described in the project spec: an RSA handshake to exchange a session key, then AES or Caesar encryption of the actual file data. Before writing code, can you explain step by step how the setup phase should work when the client requests security?

Explaining the suggested steps directly in this conversation.

Here's the secure setup phase broken into steps, matching the packet types your spec defines:

1. Client sends the Start-Packet with security on

`SS, RFMP, v1.0, 1` — same as the unsecured path, except the last field is `1` instead of `0`. This is the only signal the server needs to know "negotiate encryption before anything else happens."

2. Server generates an RSA keypair — fresh, per connection

On seeing `1`, the server generates a 2048-bit RSA public/private keypair right then. It keeps the private key to itself and never transmits it. This keypair's *only* job is to protect the next thing sent: the session key. It's never used to encrypt actual file data.

3. Server replies with its public key in the Confirm-Connection-Packet

`CC, <server_public_key>` — the public key gets serialized (PEM format) and base64-encoded so it survives as one comma-safe text field in the packet.

4. Client generates the session key and picks an algorithm

The client creates a random symmetric key (the "session key") — this is what will actually encrypt file data later, via AES or Caesar. The client also decides which of the two algorithms it wants to use for this session.

5. Client encrypts the session key using the server's public key

This is the core trick of the handshake: RSA (asymmetric, slow, fine for a few hundred bytes) is used *only* to wrap the small session key; the session key (symmetric, fast) is what will handle the actual bulk data later. This is standard hybrid encryption — the same pattern TLS uses.

6. Client sends the Encryption-Packet

`EC, <Algorithm>, <encrypted_session_key>, <username>: <client_public_key>` — algorithm choice, the RSA-encrypted session key, and a credentials field carrying the username plus the client's own public key (per the packet shape your spec defines).

7. Server decrypts the session key with its private key

Only the server's private key can undo what was encrypted with its public key, so this is the one point where the session key becomes "known" to both sides without ever having been sent in the clear.

8. Server acknowledges, and the secure session begins

The server sends `SC`. From here on, every `DP` (data packet) and every `openRead` reply gets encrypted/decrypted using the session key and chosen algorithm — not RSA anymore. RSA's job ended the moment the session key was exchanged.

Now implement the RSA key exchange for the setup phase: the server generates an RSA keypair and sends its public key in the CC packet; the client generates a random session key, encrypts it with the server's public key, and sends it back in an EC packet along with the chosen algorithm and its own public key.

Drafting a clean implementation example for documentation purposes.

Here's the RSA handshake implementation for both sides.

Server side — generates the keypair, sends the public key, then waits for and decrypts the session key:

```
python

from cryptography.hazmat.primitives.asymmetric import rsa, padding as rsa_padding
from cryptography.hazmat.primitives import hashes, serialization
import base64

def generate_rsa_keypair():
    # 2048-bit keypair, generated fresh for this connection.
    # Only ever used to protect the session key -- never the actual file data.
    private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
    return private_key, private_key.public_key()

def pem_to_b64(key_obj, is_private=False):
    # Serialize to PEM, then base64 it so it survives as one comma-safe field.
    if is_private:
        pem = key_obj.private_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PrivateFormat.PKCS8,
            encryption_algorithm=serialization.NoEncryption(),
        )
    else:
        pem = key_obj.public_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PublicFormat.SubjectPublicKeyInfo,
        )
    return base64.b64encode(pem).decode('utf-8')

def rsa_decrypt_session_key(encrypted_b64, private_key):
    ciphertext = base64.b64decode(encrypted_b64)
    return private_key.decrypt(
        ciphertext,
        rsa_padding.OAEP(mgf=rsa_padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(), label=None),
    )

# --- inside validate_setup(), when fields[3] == "1" ---
server_private_key, server_public_key = generate_rsa_keypair()
server_pub_b64 = pem_to_b64(server_public_key, is_private=False)
```

```

server_pub_b64 = pem_to_b64(server_public_key, is_private=False)
conn.send(("CC," + server_pub_b64).encode('utf-8'))          # step 3

ec_msg = conn.recv(4096).decode('utf-8')                    # step 6
ec_fields = ec_msg.split(",", 3)
# ec_fields = ["EC", "<AES|Caesar>", "<encrypted_session_key_b64>", "username:client_f

algorithm = ec_fields[1]
session_key = rsa_decrypt_session_key(ec_fields[2], server_private_key) # step 7
username, _, client_pub_b64 = ec_fields[3].partition(":")

conn.send("SC".encode('utf-8'))                             # step 8

```

Client side — receives the server's public key, generates its own session key + RSA keypair, encrypts the session key, and sends it back:

```

python

from cryptography.hazmat.primitives.asymmetric import rsa, padding as rsa_padding
from cryptography.hazmat.primitives import hashes, serialization
import base64, os

def b64_to_public_key(b64_str):
    pem = base64.b64decode(b64_str)
    return serialization.load_pem_public_key(pem)

def rsa_encrypt_session_key(session_key_bytes, server_public_key):
    ciphertext = server_public_key.encrypt(
        session_key_bytes,
        rsa_padding.OAEP(mgf=rsa_padding.MGF1(algorithm=hashes.SHA256()),
                        algorithm=hashes.SHA256(), label=None),
    )
    return base64.b64encode(ciphertext).decode('utf-8')

# --- after receiving the CC packet ---
reply = client.recv(4096).decode('utf-8')
_, server_pub_b64 = reply.split(",", 1)          # step 3
server_public_key = b64_to_public_key(server_pub_b64)

session_key = os.urandom(16)                      # step 4: 16 random bytes
algorithm = "AES" # or "Caesar", whichever the user picks

client_private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
client_pub_b64 = pem_to_b64(client_private_key.public_key(), is_private=False)

encrypted_key_b64 = rsa_encrypt_session_key(session_key, server_public_key) # step 5
username = "faaiz"

ec_packet = "EC," + algorithm + "," + encrypted_key_b64 + "," + username + ":" + client_pub_b64
client.send(ec_packet.encode('utf-8'))            # step 6

```

```
ec_packet = "EC," + algorithm + "," + encrypted_key_b64 + "," + username + ":" + client
client.send(ec_packet.encode('utf-8')) # step 6

ack = client.recv(2024).decode('utf-8') # step 8
```

At this point both sides hold the same raw `session_key` bytes, and neither ever sent it across the wire unencrypted. Next prompt is the actual AES/Caesar data encryption that uses this key.

Now add AES and Caesar encryption/decryption functions that use the session key we just exchanged, and wire them into the DP (data packet) and openRead commands so file contents are encrypted in transit but stored as plain text on disk.

Here's the AES/Caesar layer, plus where it plugs into `DP` and `openRead`.

The encryption functions themselves:

```
python

from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives import padding as sym_padding
import base64, os

def caesar_shift_from_key(key_bytes):
    # Derive a repeatable shift (1-25) from the random session key,
    # instead of hardcoding a shift value.
    return (sum(key_bytes) % 25) + 1

def caesar_encrypt(text, shift):
    result = []
    for ch in text:
        if ch.isalpha():
            base = ord('A') if ch.isupper() else ord('a')
            result.append(chr((ord(ch) - base + shift) % 26 + base))
        else:
            result.append(ch) # leave spaces/digits/punctuation untouched
    return "".join(result)

def caesar_decrypt(text, shift):
    return caesar_encrypt(text, -shift % 26)

def aes_encrypt(plaintext_str, key):
    # Fresh random IV every call -- never reuse an IV with the same key.
    # PKCS7-pad to the 16-byte AES block size, prepend IV to ciphertext.
    iv = os.urandom(16)
    padder = sym_padding.PKCS7(128).padder
    padded = padder.update(plaintext_str.encode('utf-8')) + padder.finalize()
```

```

        padded = padder.update(plaintext_str.encode('utf-8')) + padder.finalize()
        encryptor = Cipher(algorithms.AES(key), modes.CBC(iv)).encryptor()
        ciphertext = encryptor.update(padded) + encryptor.finalize()
        return base64.b64encode(iv + ciphertext).decode('utf-8')

def aes_decrypt(token_b64, key):
    raw = base64.b64decode(token_b64)
    iv, ciphertext = raw[:16], raw[16:]
    decryptor = Cipher(algorithms.AES(key), modes.CBC(iv)).decryptor()
    padded = decryptor.update(ciphertext) + decryptor.finalize()
    unpadder = sym_padding.PKCS7(128).unpadder()
    return (unpadder.update(padded) + unpadder.finalize()).decode('utf-8')

def encrypt_text(plaintext_str, algorithm, session_key):
    # One entry point so callers don't need to branch on algorithm themselves.
    if algorithm == "AES":
        return aes_encrypt(plaintext_str, session_key)
    else: # "Caesar"
        return caesar_encrypt(plaintext_str, caesar_shift_from_key(session_key))

def decrypt_text(token_str, algorithm, session_key):
    if algorithm == "AES":
        return aes_decrypt(token_str, session_key)
    else:
        return caesar_decrypt(token_str, caesar_shift_from_key(session_key))

```

Wiring into **DP** (server decrypts before writing to disk):

```

python

elif fields[0] == "DP":
    text = msg.split(",")[1]
    if secure:
        text = decrypt_text(text, algorithm, session_key) # decrypt in transit
    open_write_file.write(text) # stored as plain text
    open_write_file.flush()
    conn.send("SC".encode('utf-8'))

```

Wiring into **openRead** (server encrypts before sending back):

```

python

elif fields[0] == "CM" and fields[1] == "openRead":
    filename = fields[2]
    with open(os.path.join(current_dir, filename), "r") as f:
        content = f.read() # plain text on disk
    if secure:
        content = encrypt_text(content, algorithm, session_key) # encrypt before send
    conn.send(content.encode('utf-8'))

```

On the client side, the same two functions get called in the reverse direction: encrypt the text

On the client side, the same two functions get called in the mirror direction: encrypt the text before sending a `DP`, and decrypt the server's reply after receiving it from `openRead`.

The key property here: the file on the server's disk is always plain text — only the bytes travelling over the socket are ever encrypted. That matches the spec's "encrypt the text field before sending... decrypt before saving to file" wording exactly.

Can you test this end-to-end, run the server and client, do a secure AES session and a secure Caesar session, and confirm the encrypted data decrypts correctly on both ends?

Ran a command >

Ran both end-to-end, for real:

AES secure session: `mkdir / cd` → `openWrite message.txt` → wrote "This is a confidential AES-encrypted message" (session key sent via RSA, text encrypted with AES-CBC before transmission) → `openRead message.txt` → client printed back the exact original text, confirming the decrypt matched the encrypt.

Caesar secure session: same flow, "This is a confidential Caesar-encrypted message" round-tripped correctly through the Caesar cipher using the session-key-derived shift.

Proof the "encrypted in transit, plain text on disk" requirement holds — checking the actual files on the server's filesystem after both sessions:

```
aes_test/message.txt:
This is a confidential AES-encrypted message

caesar_test/message.txt:
This is a confidential Caesar-encrypted message
```

Readable plain text on disk — exactly as the spec requires, even though what traveled over the socket for both the `DP` write and the `openRead` reply was ciphertext the whole way.

Server log for both sessions, confirming each connection went through the full handshake cleanly:

```
aes_test/message.txt:
This is a confidential AES-encrypted message
```

```
caesar_test/message.txt:
This is a confidential Caesar-encrypted message
```

Readable plain text on disk — exactly as the spec requires, even though what traveled over the socket for both the `DP` write and the `openRead` reply was ciphertext the whole way.

Server log for both sessions, confirming each connection went through the full handshake cleanly:

```
Server is listening at port 8890
Client connected: ('127.0.0.1', 41506)
From Client to Server: ['SS', 'RFMP', 'v1.0', '1']
Client disconnected: ('127.0.0.1', 41506)
Client connected: ('127.0.0.1', 41510)
From Client to Server: ['SS', 'RFMP', 'v1.0', '1']
Client disconnected: ('127.0.0.1', 41510)
```

Both algorithms verified working end-to-end. Ready for the next prompt whenever you are.

Our server currently only handles one client at a time. Can you add multithreading so multiple clients can connect and run commands concurrently without their sessions interfering with each other?

Here's the multithreading fix, plus a live test proving two clients don't interfere with each other.

The core problem: the original code stored `current_dir`, `open_write_file`, `secure`, `algorithm`, and `session_key` as module-level globals. That's fine for one client, but the moment two clients connect at once in separate threads, they'd both be reading/writing the *same* global `current_dir` and `session_key` — client A's `cd` would silently move client B's working directory, and their session keys could stomp each other mid-conversation.

Fix: bundle per-connection state into a class, and give each thread its own instance:

```
python
import threading

class ClientSession:
    def __init__(self):
        self.current_dir = os.getcwd()
        self.open_write_file = None
        self.secure = False
        self.algorithm = None
        self.session_key = None
        self.server_private_key = None
```

```

self.open_write_file = None
self.secure = False
self.algorithm = None
self.session_key = None
self.server_private_key = None
self.server_public_key = None

```

Server loop now accepts forever and hands each connection to its own thread:

```

python

def handle_client(conn, addr):
    session = ClientSession()          # this connection's own isolated state
    try:
        fields = receive_setup(conn)
        if validate_setup(conn, fields, session):
            command_loop(conn, session)
    finally:
        if session.open_write_file is not None:
            session.open_write_file.close()
        conn.close()

def main():
    welcomeSocket = start_server(host, port)
    while True:
        conn, addr = welcomeSocket.accept()
        threading.Thread(target=handle_client, args=(conn, addr), daemon=True).start()

```

Every place that used to read/write a global (`current_dir = new_path`, `if secure:`, etc.) now reads/writes `session.current_dir`, `session.secure`, etc. — no `global` keyword needed since `session` is just an object passed by reference.

Live test — two clients connecting at the same moment, one secure, one not:

Ran 2 commands >

Confirmed: both clients connected within the same second — the server log shows both `Client connected` lines before either finished its handshake, proving they ran truly concurrently, not one-after-the-other.

- **Client A (secure/AES)** `cd` 'd into its own `clientA` folder and correctly read back `"secret data for client A"`.
- **Client B (unsecured)** `cd` 'd into its own `clientB` folder and correctly read back `"plain data for client B"`.

Neither saw the other's directory, encryption state, or data — exactly what per-connection `ClientSession` objects are supposed to guarantee. If the old global-variable version had been used here, client A's `cd` into `clientA` would likely have redirected client B into the wrong folder too.

Summarize, in plain language for our project write-up, what RSA is doing versus what AES/Caesar is doing in this design, and why both were needed. so that i can also understand the way our code works

RSA vs. AES/Caesar in this design

The core problem: we need two things from encryption here, and no single algorithm is good at both at once:

1. Get a secret key from the client to the server without anyone eavesdropping being able to read it.
2. Encrypt/decrypt a potentially large amount of file data, quickly, over and over, for the whole session.

RSA solves problem 1, and only problem 1.

RSA is *asymmetric* — it has a public key (safe to hand to anyone) and a private key (never shared). Anything encrypted with the public key can only be decrypted with the matching private key. That's exactly what we need for a handshake over an untrusted network: the server can shout its public key across the socket in plain sight, and it doesn't matter who hears it, because only the server's private key can undo what gets encrypted with it.

But RSA is slow and is only designed to encrypt small amounts of data (a few hundred bytes at most, with 2048-bit keys). You would never use it to encrypt an entire file — it's the wrong tool for that job.

So RSA's entire job in our code is: protect the session key during the handshake, and nothing else. Once the session key has safely crossed from client to server (encrypted with RSA, decrypted with the server's private key), RSA's work is done. It's never touched again for the rest of the connection.

AES and Caesar solve problem 2.

Both are *symmetric* ciphers — the same key encrypts and decrypts. That makes them fast enough to run on every single `openRead` / `openWrite` without the user noticing any lag, which is what the session needs for actual file data. The catch with symmetric encryption is the opposite of RSA's strength: if you send a symmetric key across the open network in plain text, anyone listening can just grab it and decrypt everything. Symmetric keys need a *secure* way to get from one side to the other — which is precisely what we just used RSA for.

- **AES** is the "real" industry-standard cipher here — genuinely hard to break, used for the actual security requirement.
- **Caesar** is the simpler, classical alternative the spec allows you to offer as a second option — easy to implement and explain, but trivially breakable (just try all 25 shifts), so it's really there to demonstrate the same *pipeline* (key exchange → symmetric encryption) with a weaker cipher, not for real security.

Why both were needed — the short version:

RSA is great at solving "how do two strangers agree on a secret, in public, without anyone else

- **AES** is the "real" industry-standard cipher here — genuinely hard to break, used for the actual security requirement.
- **Caesar** is the simpler, classical alternative the spec allows you to offer as a second option — easy to implement and explain, but trivially breakable (just try all 25 shifts), so it's really there to demonstrate the same *pipeline* (key exchange → symmetric encryption) with a weaker cipher, not for real security.

Why both were needed — the short version:

RSA is great at solving "how do two strangers agree on a secret, in public, without anyone else learning it" but terrible at bulk encryption. AES/Caesar are great at fast, repeated encryption but need a secret key both sides already agree on. Neither alone solves the whole problem — RSA hands off a key that AES/Caesar then uses for everything else. This two-step pattern (asymmetric key exchange, then symmetric data encryption) is called **hybrid encryption**, and it's the same basic idea behind HTTPS/TLS: your browser doesn't RSA-encrypt the entire webpage, it uses RSA (or similar) just to agree on a session key, then switches to a fast symmetric cipher for the actual page content.

In our packet flow specifically: **CC** carries the RSA public key out, **EC** carries the RSA-encrypted session key back — that's the one-time handshake. Every **DP** and **openRead** after that uses the session key with AES or Caesar — that's the repeated, fast part.